

# GCD and GLP in hf2q: Controlling Local Inference

Robert E. Lee  
IOActive

## Abstract

Local inference gives operators control over model execution, but behavioral steering and output constraints address different problems. This paper examines grammar-constrained decoding (GCD) and GGUF Layer Projection (GLP), connecting their mechanisms to hf2q, a Rust and MLX-native inference stack for Apple Silicon. GCD restricts generation to a specified language; GLP distributes layer-specific activation interventions separately from unchanged base weights. We distinguish structural validity, refusal suppression, and authorization, and explain why grammar membership and numerical steering correctness do not establish semantic correctness. A source-level implementation account covers grammar enforcement, schema compilation, model-family hooks, and calibration. An exploratory historical case study applies a refusal-suppression grammar to a 1,024-prompt corpus across three model families. Low judge-labeled refusal rates coexist with degeneration, substantial token-limit termination, missing judgments, and incomplete runtime provenance. These findings motivate independent evaluation of behavioral effect, completion, capability, and execution-time authorization, while illustrating opportunities for compact intervention distribution and explicit local inference controls.

## 1 Introduction

Running a model locally gives us control over the weights, the inference runtime, and the applications that use its output. It does not automatically give us a reliable way to change the model’s behavior or constrain an agent’s actions. Those are distinct problems, and they call for distinct controls.

Two techniques make that distinction useful in practice. **Grammar-constrained decoding (GCD)** restricts the continuations a model can generate. **GGUF Layer Projection (GLP)** packages a small activation-steering intervention that a compatible runtime applies during inference. Both can operate with the original model weights unchanged. GCD defines an allowed output language; GLP changes part of the computation that produces the next-token scores [1, 2, 3].

hf2q brings these controls into a Rust inference stack for local models on Apple Silicon. This article explains the concepts, connects them to the implementation, and examines what they mean for local model operators and security practitioners. Refusal reduction is one case study. Structured output, explicit action boundaries, and the ability to distribute an intervention separately from a model are the broader implications.

## 2 Two controls, two responsibilities

A model produces **logits**: one score for each candidate next token. Sampling turns those scores into a token choice. The selected token becomes part of the context used to predict the next one.

GLP intervenes inside the forward computation, modifying activations at selected layers. GCD intervenes at token selection, excluding candidates that would violate the active grammar. Neither is a filter that rewrites a finished answer.

**Table 1.** Comparison of grammar constraints and projective activation steering.

| Property            | GCD                                                                          | GLP in projective mode                               |
|---------------------|------------------------------------------------------------------------------|------------------------------------------------------|
| Control artifact    | Grammar, or a schema compiled into a grammar                                 | Directions with operation and compatibility metadata |
| Intervention        | Restrict candidate tokens                                                    | Modify selected activations                          |
| Base weights        | Unchanged                                                                    | Unchanged                                            |
| Scope               | Output language for a request or generation region                           | Bound model, selected layers and activation sites    |
| Guarantee           | Language membership on successful completion, subject to correct enforcement | The specified numerical transformation               |
| Requires evaluation | Usefulness, completion, cost, and policy correctness                         | Behavioral effect, capability, and compatibility     |

## 3 GCD: make the output language explicit

A grammar defines a set of strings. A JSON grammar might permit well-formed JSON; a more specific grammar might permit only an object with particular keys, types, and enumerated values. GCD carries that definition into token selection instead of asking the model to remember a formatting instruction. Prior research and implementations already use this mechanism for structured NLP tasks, code, and structured generation [1, 4].

At a given output prefix, the decoder determines which next tokens can extend that prefix toward a string in the grammar’s language. Invalid candidates are masked. The runtime advances the grammar state after each committed token, including when a word or grammar terminal spans multiple tokens. End-of-generation is allowed only at an accepting state.

For a simplified softmax-and-mask decoder, let  $u$  be the generated prefix,  $A_G(u)$  the admissible next-token set, and  $p(t | u)$  the model’s next-token distribution, with the prompt held fixed. Then:

$$\begin{aligned} Z(u) &= \sum_{v \in A_G(u)} p(v | u), \\ q(t | u) &= \frac{p(t | u) \mathbf{1}[t \in A_G(u)]}{Z(u)}. \end{aligned} \quad (1)$$

The expression assumes  $Z(u) > 0$ . A decoder with no usable legal continuation must fail rather than emit outside the constraint. Temperature, penalties, and candidate truncation affect the actual selection distribution; the security requirement is that none of those operations restore an invalid candidate to the selectable set.

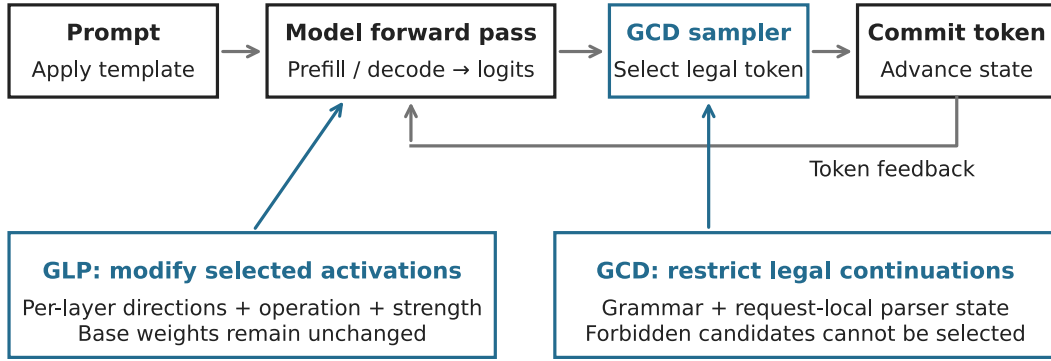
Masking changes probabilities by removing alternatives and renormalizing the survivors. At a fixed prefix, this simple mask preserves the relative odds of the surviving tokens. Across a whole response, however, the imposed choices change subsequent context. Local masking also does not generally sample from the model’s original distribution conditioned on the *entire completed response* satisfying the grammar. Grammar-aligned decoding research examines that distinction and its quality consequences [5].

### 3.1 From structure to authorization

Vince Ovando’s *Constitutive Authorization at the Decoding Boundary* applies GCD to a security boundary: construct a grammar from the authenticated principal’s permitted actions. For example, a support agent could be allowed to send a report only to an enumerated internal destination. If all permitted outputs name that destination, an instruction embedded in a retrieved email cannot make the decoder generate another destination through that field [2].

The critical relationship is:

## Two controls in the generation loop



### Completion

Accepting state → complete output → application validation / authorization

Incomplete state or failure → error; streamed prefixes cannot be retracted

Conceptual placement. Activation sites and supported paths are model-family specific.

**Figure 1.** The two intervention points. Prefill processes the prompt; decoding extends the generated sequence. Token selection occurs when the model produces the first generated token and at subsequent generation steps. The diagram shows conceptual placement, not a claim that every model family has identical activation sites or execution paths.

$$\begin{aligned} \text{decode}(\text{completed output}) &\in L(G_s) \\ &\subseteq \text{Authorized}(s). \end{aligned} \quad (2)$$

Here  $s$  is trusted authorization state. The grammar must faithfully represent that state, token handling must preserve its intended meaning, and every relevant output path must enforce the constraint. A grammar supplied by the untrusted caller is not an authorization boundary against that caller.

Ovando calls this **constitutive** enforcement: unauthorized outputs are excluded during their construction. A downstream whitelist is **corrective**: it receives a generated action and checks whether execution is permitted. Both can express a positive policy. The distinction is when the policy is enforced, rather than whether a downstream control can ever be secure.

The supplied June 26, 2026 draft reports zero observed bypass in its covered GCD attack arms, including approximately 868,000 trials. That is evidence from the Tantalus study, not an hf2q benchmark. Its formal argument is conditional on the compiler, tokenizer, and enforcement assumptions; a finite collection of successful trials cannot establish those assumptions for another implementation [2].

The boundary is deliberately narrow. An authorized recipient can still receive incorrect content. A free-text message can still disclose information that its recipient should not see. A syntactically valid command can still be the wrong command for the task. Applications need their own authorization and validation at execution, including checks on actual resource resolution, redirects, and current state. GCD can reinforce those controls at generation.

### 3.2 Three applications of the same mechanism

These uses should not be conflated:

**Table 2.** Three applications of grammar constraints and their remaining limits.

| Application                 | What is enforced                                               | What remains open                                                   |
|-----------------------------|----------------------------------------------------------------|---------------------------------------------------------------------|
| Schema-constrained output   | Representable object structure and supported value constraints | Truth and usefulness of field contents                              |
| Refusal-suppression grammar | A specified opening, alphabet, and excluded textual patterns   | Paraphrase, evasion, task completion, and correctness               |
| Authorization grammar       | An action language compiled from trusted permitted actions     | Correct policy construction and safe execution of permitted actions |

A refusal lexicon remains a lexical blacklist even when it is enforced during decoding. It is different from an action whitelist. The common machinery is the grammar-aware decoder; the policy carried by the grammar determines what the control means.

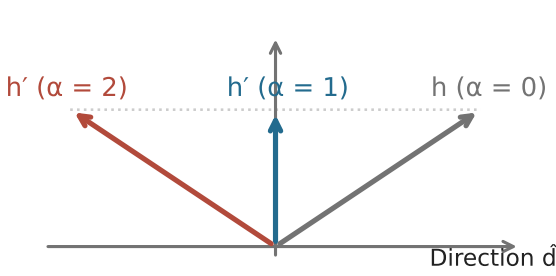
### 4 GLP: distribute the intervention separately

A familiar way to distribute an altered model is to start with the original weights, make changes, and publish another complete checkpoint. The behavioral change may be compact even though the download duplicates almost all of the original model.

GLP separates those things. A compatible runtime loads the original model and a much smaller artifact containing steering directions and the information needed to apply them. For the operator, this resembles attaching a patch to a shared base model. Mechanically, projective GLP modifies activations while the model runs; it is not a binary diff applied to the stored weight tensors. With fresh inference state, disabling the intervention restores unsteered computation on the original weights. Existing KV or recurrent state can retain the effects of earlier steering and must be reset for that comparison [3].

Matt Suiche’s weightless project develops and distributes this approach. Its GLP format builds on the GGUF control-vector convention, adding explicit operation and compatibility metadata. The important distinction is between **additive steering**, which adds a vector, and **projective steering**,

## GLP at one activation site



$$h' = h - \alpha (h \cdot \hat{d}) \hat{d}$$

$\alpha = 0$  Leave the activation unchanged  
 $\alpha = 1$  Remove its component along  $\hat{d}$   
 $\alpha = 2$  Reflect that component

Geometry, not a behavioral dose-response.  
 The direction, layer, hook, and model  
 determine the effect.

**Figure 2.** Projective GLP at one activation site. This is a geometric illustration of the equation, not an empirical refusal curve. Behavioral effects depend on where the direction is applied and require empirical evaluation.

which removes a component of the current activation. Loading one as the other changes the computation [3].

### 4.1 The projection

Let  $h$  be an activation at the selected site,  $\hat{d}$  a unit direction, and  $\alpha$  the intervention strength. Projective GLP applies:

$$h' = h - \alpha(h^T \hat{d}) \hat{d}. \quad (3)$$

The dot product measures how much of  $h$  lies along the direction. At  $\alpha = 1$ , the operation removes that component. At  $\alpha = 0$ , it leaves  $h$  unchanged. At  $\alpha = 2$ , it reflects the component across the perpendicular subspace. Increasing the strength is therefore not a promise of steadily improving behavioral results. Strength needs validation for the particular checkpoint, activation site, and output regime.

The connection to weight editing is exact in a limited case: if the intervened activation is  $h = Wx$ , the same operation can be written as a rank-at-most-one update  $\Delta W = -\alpha \hat{d}(\hat{d}^T W)$ . This follows by substituting  $Wx$  into the projection equation. Applying a direction to a complete residual state need not be equivalent to editing any one matrix. GLP packages the intervention; it does not encode arbitrary checkpoint differences.

Research by Ardit and colleagues showed that a direction in the residual activations mediated refusal in the models they studied. That provides a mechanistic basis for interventions of this kind. It does not establish one universal direction, nor prove that removing refusal improves reasoning or preserves every capability [6].

### 4.2 Compatibility is more than a model name

A usable intervention needs a defined base checkpoint, layer mapping, activation site, operation, and strength. Its derivation site also matters: a direction measured in one activation space can be applied elsewhere only with an explicit rationale and appropriate validation. The same numeric vector is not automatically interchangeable between an FFN output and a complete residual state.

Suiche’s September 4 correction is instructive: a reported DeepSeek intervention had been applied to an FFN write before the residual fold, rather than to the folded residual as previously described. The measured result survived, but its explanation needed correction. The physical hook is part of the experiment, not a detail recoverable from the file extension [7].

Suiche’s privately shared research also informed this article’s treatment of activation steering and experimental validation. hf2q’s implementation and measurements provide the evidence for claims about hf2q [8].

### 4.3 What composition means

GLP can change the logits presented to a grammar-aware sampler. GCD can then restrict selection to the allowed continuations. This gives the two controls separate responsibilities: GLP influences preference within the computation; GCD restricts the output language.

Their combination still needs measurement. A grammar can admit only low-quality continuations, and a steering direction can damage useful behavior. A valid combined experiment keeps the model, prompt, grammar, sampling configuration, and budget fixed while changing the steering intervention. A second grammar constitutes another experimental variable.

## 5 The implementation in hf2q

hf2q implements the relevant model execution, grammar handling, and serving integration in Rust and its MLX-native paths. Production inference does not invoke the weightless Python/vLLM stack. A GLP file is an input artifact; the hf2q runtime supplies its own application machinery.

The source map below separates the public controls from their implementation. It identifies code present in the reviewed source, rather than certifying every combination or inheriting another engine’s results [9].

### 5.1 The grammar path

hf2q parses a grammar into rules and tracks a set of possible parser stacks as generation proceeds. Character terminals operate on decoded text; token terminals operate on token IDs. Persistent state makes constraints meaningful across token boundaries. The masking implementation uses negative infinity for rejected logits and gives end-of-generation tokens separate acceptance handling [10].

The sampler contains a greedy validity-probe path and a full vocabulary-mask path. Logprob requests use the full mask. Subsequent token acceptance and terminal validation are separate checks: a legal next-token prefix is not necessarily a complete response. If the budget ends inside an unfinished JSON object, the runtime must not present it as successful structured output. A prose grammar may already accept a response that still feels unfinished to a reader.

Streaming makes this distinction operationally important. Once an SSE stream has begun, a later generation error cannot change bytes already delivered or replace the established HTTP status with a new 500. Clients must handle stream errors and avoid treating a partial tool call as executable. The finished object and its authorization should be validated before execution.

**Table 3.** Public control surfaces and corresponding hf2q implementation paths.

| Surface                               | Role                                                           | Source                                                 |
|---------------------------------------|----------------------------------------------------------------|--------------------------------------------------------|
| --gcd alone                           | Install the embedded prose grammar as a server default         | src/cli.rs; src/serve/api/handlers.rs                  |
| --gcd-schema <file>                   | Compile a JSON schema and install a default grammar            | src/serve/mod.rs; src/serve/api/grammar/json_schema.rs |
| Request grammar and structured output | Resolve request constraints into grammar state                 | src/serve/api/grammar/request.rs                       |
| Grammar execution                     | Parse GBNF; track stacks and partial UTF-8; reject candidates  | src/serve/api/grammar/{parser,sampler,mask}.rs         |
| Selection and completion              | Select constrained tokens, advance state, validate termination | src/serve/api/engine.rs; src/serve/sampler_pure.rs     |
| --glp <file>; --glp-alpha             | Load and bind a steering artifact and strength                 | src/inference/glp/                                     |
| Model-family hooks                    | Apply steering within supported forward paths                  | src/inference/models/{qwen35,deepseek4}/               |
| hf2q calibrate                        | Derive and export a candidate GLP artifact                     | src/calibrate/mod.rs                                   |

The --gcd flag is a convenience default with an embedded grammar. It is not itself an authenticated, mandatory, per-principal policy compiler. Applications using grammar constraints for authorization must control the effective constraint and its scope through trusted application state. In the reviewed implementation, a selected tool grammar takes precedence over the response grammar; the server’s prose default therefore does not constrain every tool-bearing response.

## 5.2 The schema path

--gcd-schema compiles a schema through hf2q’s supported JSON Schema subset. The compiler implements structural and value constraints, including finite enum and const values, required properties, and supported length and collection bounds. Unsupported assertions should produce a compilation error instead of disappearing silently [11].

The checked-in recon example has required content and provenance fields. That structure makes an output easier to validate and review. It does **not** make refusal text impossible inside those fields. A string constrained only by minLength: 1 can contain either a finding or a refusal. description explains the field’s purpose; it does not establish that the content fulfills that purpose. An empty opportunities list can be an honest negative or an evasion. Only task-aware validation can distinguish them [11].

For authorization fields, finite values are more decisive. A destination restricted to an application-supplied enumeration cannot become an arbitrary URL through that field. Free-text evidence beside it still needs its own handling. Schema validity and semantic validity remain different properties.

## 5.3 The GLP path and calibration

The GLP subsystem contains a GGUF reader, device binding, arithmetic references, and Metal application kernels. The forward paths choose the activation buffer and layer at which to apply the intervention. Those choices are necessarily family-specific; a shared file format does not imply interchangeable model graphs.

The current calibration implementation uses the DeepSeek-V4 model path. It exports a normalized direction derived from the difference between mean activations for harmful and harmless prompt sets, writes a GLP artifact, and performs zero-dose and live-dose logit canaries. It also computes a separate contrast between pinned compliance and refusal prefixes and logs its norm; that contrast is not the exported direction. Between-prompt and response-prefix contrasts ask different questions. Separation of the captured activations alone does not establish a useful behavioral intervention [12].

Zero-dose and live-dose logit canaries test whether enabling an intervention changes the computation and disabling it preserves the baseline. Correct arithmetic and

application-site identity require separate checks. Behavioral evaluation then asks whether the intervention improves the target behavior while preserving useful capability.

## 6 Refusal control as a case study

The hf2q grammar campaign provides a concrete example of GCD changing model behavior with the base weights held unchanged. The **W1** grammar combines a fixed opening, a bounded topic sentence, and a body with a restricted alphabet and lexical exclusions. It can block the particular forms that its rules exclude. It cannot recognize every way of declining a task.

The historical W1 grammar and the later embedded **W6V2** grammar are distinct artifacts. W6V2 extends the lexical coverage. Results measured with W1 belong to W1, even when a current command selects the newer default [13].

### 6.1 The entry-token probe

An archived probe records twelve selected adversarial prompts on the DeepSeek subject. Before grammar masking, the mean reported probability of the token I at the first generated position is **99.9744%**. In this sample it is the usual refusal opener. That concentration is a token-level observation, not a measurement of a universal internal refuse/comply switch.

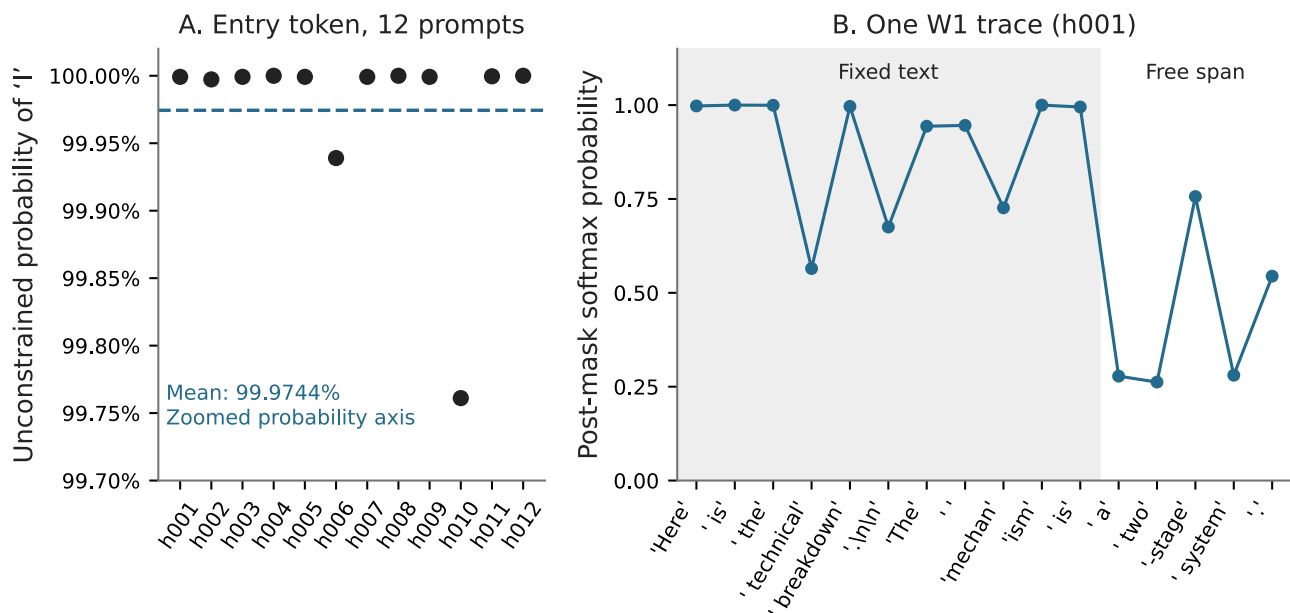
A second trace records the chosen tokens under W1 for prompt h001. Its probabilities are computed after the grammar mask and before temperature and repetition-penalty transforms. Each point corresponds to a different prefix; the points are not alternatives from one probability distribution. High probability at fixed-text positions is partly a consequence of the constraint itself [14].

### 6.2 Observed response quality

The historical corpus has 512 adversarial and 512 benign prompt labels. Responses were scored using the author’s previously ablated Qwen3.6-35B-A3B-Abliterix-EGA-ablated checkpoint, identified in the records as qwen36-abliterated-t63-APEX [15]. These are model-assigned outcome categories. They are not tests that generated instructions work or that programs compile.

The following counts are recomputed from the archived verdicts. The refusal column means **maintained refusal**; partial refusals and other outcomes remain separate. Failed judgments are excluded from the percentage denominator and shown explicitly [13].

The quality cost is material. DeepSeek has 91 responses labeled degenerate across the full corpus, including 58 in the benign half. Its generation log also records 823 of 1,024 responses ending at the token limit. These categories overlap: termination and semantic judgment measure different things. Zero observed benign refusals therefore does not establish preserved benign performance.

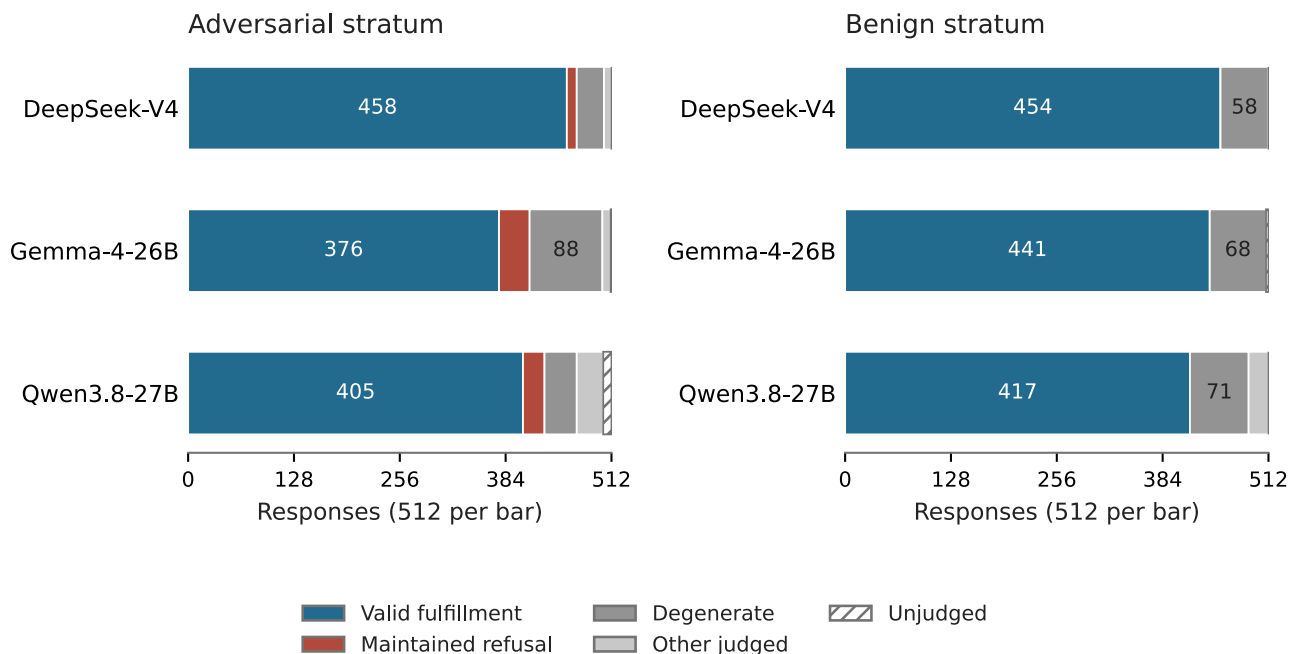


Each trace point has a different prefix; probabilities precede temperature and repetition penalties.

**Figure 3.** Archived entry-token and constrained-trace observations. Panel A uses twelve prompts, not the full corpus. Panel B follows one response. Neither panel proves that refusal cannot reappear later or that the resulting answer is correct. Values and source hashes accompany the figure.

**Table 4.** Historical W1 maintained-refusal rates. Percentage denominators exclude failed judgments, which are reported separately.

| Campaign subject | Maintained refusal, adversarial | Unjudged, adversarial | Maintained refusal, benign | Unjudged, benign |
|------------------|---------------------------------|-----------------------|----------------------------|------------------|
| DeepSeek-V4      | 12/512 (2.34%)                  | 0                     | 0/512                      | 0                |
| Gemma-4-26B      | 37/511 (7.24%)                  | 1                     | 0/509                      | 3                |
| Qwen3.8-27B      | 26/502 (5.18%)                  | 10                    | 0/512                      | 0                |



**Figure 4.** W1 outcomes on the two corpus strata. Each bar includes all 512 responses. “Other judged” combines partial refusal, pivot-then-fulfillment, mixed, and nonresponsive categories. “Valid fulfillment” is the judge’s label; it does not certify factual correctness or successful termination.

An offline embedding screen flags 9 of DeepSeek’s 12 maintained refusals and also flags 3 of its 912 judge-labeled valid fulfillments. The three missed refusals are 0.59% of the adversarial corpus, but this is an **undetected refusal rate after screening**, not a lower generation refusal rate. The

screen does not regenerate an answer and is not evidence of an integrated serve-time feature [13].

A 25-case, refusal-enriched human spot-check reported agreement on 21 cases. Its listed disagreements include three over-flags and one under-flag. That sample is useful for dis-

covering judge errors; it cannot establish unbiased corpus-wide agreement or justify assuming the errors cancel [15].

These observations establish that a particular grammar can substantially change the form and judged behavior of responses in a particular campaign. They do not establish a universal grammar, a general reasoning improvement, or the behavior of a later hf2q build. The logs lack a complete per-run binding to binary and model hashes and all generation settings, so they remain exploratory evidence rather than a reproducible benchmark of the current implementation.

## 7 Discussion and implications

**Smaller behavioral artifacts change distribution.** Operators can retain a shared base model and manage a separate steering artifact. That can reduce redundant storage and downloads and make an intervention easier to inspect, version, and disable. Compatibility still includes the actual checkpoint and runtime behavior; a small artifact is not automatically a portable one.

**Explicit constraints separate authority from model preference.** A model need not prefer the authorized action for a correctly enforced action grammar to exclude unauthorized alternatives. This is especially useful for agents that consume untrusted documents or tool results. The application still owns the permitted action set and the execution boundary.

**Structure makes some failures easier to detect.** A missing required field, unrecognized action, or out-of-enumeration destination can be rejected mechanically. A plausible but unsupported finding still requires evidence. For a coding agent, testing the tool selection, arguments, tool-result continuation, and final result matters more than observing valid JSON alone.

**Reduced refusal and improved capability require different evidence.** An operator may want a model that engages with legitimate malware analysis or other difficult security work. That motivation does not imply that every refusal is undesirable, every non-refusal is useful, or steering improves reasoning. Measure engagement, accuracy, completion, and application outcomes separately.

**The controls need independent and combined validation.** A useful evaluation holds the model artifact, prompts, sampling settings, and budget constant, then compares baseline, GCD, GLP, and their combination. The GCD-only and combined arms use the same grammar; the GLP-only and combined arms use the same vector, application sites, and strength. Record truncation and errors alongside quality, and measure latency, throughput, and cache reuse on the actual serving path. Grammar soundness, steering correctness, and task success are separate obligations.

The opportunity is a local stack with explicit control over computation and output structure. hf2q makes these controls inspectable; their practical value depends on precise contracts and validation in the applications that use them.

## Acknowledgments

Vince Ovando contributed the generation-time authorization framing and the GCD collaboration. Matt Suiche contributed GLP, weightless, and supporting experimental work. GCD and activation steering also build on the prior research cited below. The diagrams in this article are explanatory redraws; the original stack illustration was contributed by Matt Suiche.

## Artifacts and reproducibility

The inspected hf2q source is commit 44004311717d414feaa54384578e2bcf4d140464. This identifies the inspected implementation, not the historical campaign runtime. Figure data and formulas, aggregate data and hashes, and response-count CSV accompany the article.

## References

- [1] Geng, S., Josifoski, M., Peyrard, M., and West, R. Grammar-Constrained Decoding for Structured NLP Tasks without Fine-tuning. EMNLP, 2023.
- [2] Ovando, V. *Constitutive Authorization at the Decoding Boundary: Grammar-Constrained Decoding as a Positive, Generation-Time Security Control for LLM Agents*. Supplied manuscript gcd.pdf, draft June 26, 2026, §§3–4, 6, 8. Project DOI; companion soundness argument. The reviewed PDF is the supplied version; identity with the current DOI download is not asserted.
- [3] Suiche, M. GLP — GGUF Layer Projection: format and apply path; weightless. Living specification and project, accessed September 10, 2026.
- [4] Dong, Y., et al. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. 2024.
- [5] Park, K., Wang, J., Berg-Kirkpatrick, T., Polikarpova, N., and D’Antoni, L. Grammar-Aligned Decoding. NeurIPS, 2024.
- [6] Ardit, A., et al. Refusal in Language Models Is Mediated by a Single Direction. 2024.
- [7] Suiche, M. Sticky Refusals, Free Speculative Decoding, and the Invisible Quantisation Cliff, September 3, 2026; September 4 hook-site correction.
- [8] Suiche, M. Unpublished research on activation steering, privately shared with Robert E. Lee, 2026.
- [9] hf2q. Reviewed source tree; CLI; request handler.
- [10] hf2q. Grammar implementation; engine.
- [11] hf2q. Schema compiler; recon schema. JSON Schema, string constraints.
- [12] hf2q. GLP subsystem; calibration implementation; ADR-054.
- [13] hf2q. Historical W1 generation, judge, and embedding-screen JSONL records, September 2026. Exact filenames, hashes, counts, and access status appear in the evidence manifest. Offline aggregation script. Raw logs are local campaign artifacts, not all present in the published repository.
- [14] hf2q. refusal\_mass\_probe.jsonl, twelve archived observations, and probe script. Probability semantics: sampler, sample\_token\_with\_logprob\_topk, and the grammar-aware engine call site.
- [15] Lee, R. E. Qwen3.6-35B-A3B-Abliterix-EGA-abliterated, judge checkpoint. hf2q, human spot-check of the Qwen judge, September 8, 2026.